

The API documentation description

V1.0.0 version

Catalogue

The API documentation description	1
Colophon	4
1. Summary	5
2. Reader / writer interface details	6
2.1 Reader connection	6
2.1.1 Reader initialization interface (Create (ReaderType commType, Channels channel, String arg1, int arg2))	6
2.1.2 Reader / writer connection interface (Connect())	6
2.1.3 Reader and writer disconnect the interface (Disconnect())	7
2.2 Read and write parameter	8
2.2.1 Get the firmware version (GetFirmwareVersion())	8
2.2.2 Obtain the operating temperature (GetReaderTemperature())	8
2.2.3 Read Gpio1,2 levels (ReadGpioValue())	8
2.2.4 Set Gpio3,4 levels (WriteGpioValue (GpioPin gpio))	9
2.2.5 Set the buzzer status (SetBeeperMode (BeeperMode mode))	9
2.2.6 Set the module wave rate (SetUartBaudrate (int baudrate))	10
2.2.7 Set the Reader 485 Address (SetReaderAddress (int newAddress))	10
2.2.8 Set Reader Identification Identification (SetReaderIdentifier (string identifier)) ..	11
2.2.9 Get the reader identification identity (GetReaderIdentifier())	11
2.2.10 Reboot Reader (Reset ())	11
2.2.11 Setting up the current working antenna (SetWorkAntenna (Antenna antennaId)) ..	12
2.2.12 Obtain the current working antenna (GetWorkAntenna())	12
2.2.13 Measurement of antenna port echo loss (GetRfPortReturnLoss (Frequency freqParameter))	13
2.2.14 Set the antenna output power (SetOutputPower (int [] rfPowers))	13
2.2.15 Obtain the antenna output power (GetOutputPower (bool is16Ants))	14
2.2.16 Set the echo loss threshold (SetAntConnectionDetector (int detectorSensitivity)) ..	14
2.2.17 Obtain the echo loss threshold value (GetAntConnectionDetector())	14
2.2.18 Set the Impinj Monza Lab Quick Read TID Function (SetAndSaveImpinjFastTid (bool useFastTid))	15
2.2.19 Get the Impinj Monza tag Quick Read TID Function (GetImpinjFastTid())	15
2.2.20 Set the RF frequency spectrum (SetFrequencyRegion (FrequencyRegion region)) ..	16
2.2.21 Acquisition of the RF frequency spectrum (GetFrequencyRegion())	16
2.2.22 Set up the RF Link (SetRfLinkProfile (int value))	17
2.2.23 Get the RF Link (GetRfLinkProfile())	17
2.2.24 Set up the E710 RF Link (E710SetRfLinkProfile (E710Profile e710Profile))	18
2.2.25 Obtain the E710 RF Link (E710GetRfLinkProfile())	18
2.3 Disc storage parameters	20
2.3.1 Set disk parameters (setInventoryCfg (InventoryCfg inventoryCfg))	20
2.3.2 Custom drive mode (CustomizedSessionTargetInventory (CustomizedSessionTargetInventoryCfg cfg))	21
2.3.3 Quick Drive-Mode (FastSwitchAntInventory (FastSwitchAntInventoryCfg cfg)) ..	21
2.4 Label operation	23

2.4.1 Set the selected label (SetAccessEpcMatch (EpcMatch epcMatch))	23
2.4.2 Get the selected label (GetAccessEpcMatch())	23
2.4.3 Read label (Read (MemBank memBank, int wordAddr, int wordCnt, byte [] accessPwd))	24
2.4.4 Read the label for multiple regions (Read(int reservedAddr,int reservedLen,int tidAddr,int tidLen,int userAddr,int userLen,byte [] accessPwd,Session session,Target target,ReadMode readMode,int timeout))	24
2.4.5 Write a label (Write (byte [] accessPwd, MemBank membank, int wordAddr, int wordCnt, byte [] writeData))	25
2.4.6 Block (BlockWrite (byte [] accessPwd, MemBank membank, int wordAddr, int wordCnt, byte [] writeData))	25
2.4.7 Lock label (Lock (byte [] accessPwd, LockMemBank membank, LockType lockType))	26
2.4.8 Destroy tags (Kill (byte [] killPwd))	27
2.5 Error code	27

Colophon

version number	Modify the log	Modify the time	modifier	remarks
V1.0.0	The initial version	2022.10.13	Xysh	

1. Summary

In RFID_API_ver1.dll, the interface of RFID reader connection, inventory, and label read and write operations is provided, to meet the users' secondary development needs, and to reduce the need for in-depth understanding of the relevant communication protocol requirements in the development. Interface data return, mainly through the commission way, the results back to the upper layer application, so as to achieve the synchronous sending of instructions, asynchronous return.

2. Reader / writer interface details

2.1 Reader connection

2.1.1 Reader initialization interface (Create (ReaderType commType, Channels channel, String arg1, int arg2))

function name	Reader Create(ReaderType commType, Channels channel, String arg1, int arg2)
parameter declaration	commType: connection type: (Serial, TCP); Channel: Reader and writer type: (one single-port, four four-port, eight eight-port, sixteen sixteen-port); Arg1: string slogan in Serial, IP address in Tcp; Arg2: Baud rate in Serial, and port number in Tcp;
returned specification	Success: Return of the reader instance; Failed: received the exception via Exception;

2.1.2 Reader / writer connection interface (Connect())

function name	Void Connect()
parameter declaration	not have
returned specification	Failed: received the exception via Exception;

2.1.3 Reader and writer disconnect the interface (Disconnect())

function name	Void Disconnect()
parameter declaration	not have
returned specification	Failed: received the exception via Exception;

2.2 Read and write parameter

2.2.1 Get the firmware version (GetFirmwareVersion())

function name	Void GetFirmwareVersion()
parameter declaration	not have
returned specification	Success: throughThe FirmwareVersionEventArgs parameter in the FirmwareVersionCallback delegate event returns, the Major primary version number, and the Minor secondary version number.

2.2.2 Obtain the operating temperature (GetReaderTemperature()))

function name	Void GetReaderTemperature()
parameter declaration	not have
returned specification	Success: throughThe ReaderTemperatureEventArgs parameter in the ReaderTemperatureCallback delegate event returns, the ReaderTemperature Celsius temperature.

2.2.3 Read Gpio1,2 levels (ReadGpioValue()))

function	Void ReadGpioValue ()
-----------------	-----------------------

name	
parameter declaration	not have
returned specification	Success: throughThe GpioPinsEventArgs parameter in the GpioPinsCallback delegate event returns, gpios. ID indicates the gpio angle, gpio. Is the high high or low level.

2.2.4 Set Gpio3,4 levels (WriteGpioValue (GpioPin gpio))

function name	Void WriteGpioValue (GpioPin gpio)
parameter declaration	GpioPin: The gpio contains two attributes, id means the gpioId number to control, High indicates the level, true is high, and false is low
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.5 Set the buzzer status (SetBeeperMode (BeeperMode mode))

function name	Void SetBeeperMode (BeeperMode mode)
parameter declaration	BeeperMode: mode is enumeration, 0x00 is quiet, 0x01 is ringing after storage, and 0x02 is for each label read.
returned	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

specification	
---------------	--

2.2.6 Set the module wave rate (SetUartBaudrate (int baudrate))

function name	Void SetUartBaudrate (int baudrate)
parameter declaration	baudrate: Baud rate of the serial port; R2000 module support: 115200,38400; E710 module support: 115200,38400;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.7 Set the Reader 485 Address (SetReaderAddress (int newAddress))

function name	Void SetReaderAddress (int newAddress)
parameter declaration	newAddress: 485 Address;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.8 Set Reader Identification Identification (SetReaderIdentifier (string identifier))

function name	Void SetReaderIdentifier (string identifier)
parameter declaration	identifier: 12 bytes long;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.9 Get the reader identification identity (GetReaderIdentifier())

function name	Void GetReaderIdentifier ()
parameter declaration	not have;
returned specification	pass throughThe ReaderIdentifierEventArgs parameter in the ReaderIdentifierCallback delegate event returns, with the parameter ReaderIdentifier expressed.

2.2.10 Reboot Reader (Reset ())

function name	Void Reset ()
--------------------------	---------------

parameter declaration	not have;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.11 Setting up the current working antenna (SetWorkAntenna (Antenna antennaId))

function name	Void SetWorkAntenna (Antenna antennaId)
parameter declaration	antennaId: For the enumeration, the value range (0x00-0x0F) represents the antenna 1-16;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.12 Obtain the current working antenna (GetWorkAntenna()))

function name	Void GetWorkAntenna ()
parameter declaration	not have;
returned	pass throughThe WorkAntennaEventArgs parameter in the WorkAntennaCallback delegate event returns, and the parameter ant indicates the current working antenna.

specification	
---------------	--

2.2.13 Measurement of antenna port echo loss (GetRfPortReturnLoss (Frequency freqParameter))

function name	Void GetRfPortReturnLoss (Frequency freqParameter)
parameter declaration	Frequency: freqParameter is the enumeration, value range (0x00-0x3B) standard frequency point;
returned specification	pass throughThe RfPortReturnLossEventArgs parameter in the RfPortReturnLossCallback delegate event returns, and the parameter Loss represents the echo loss value.

2.2.14 Set the antenna output power (SetOutputPower (int [] rfPowers))

function name	Void SetOutputPower (int[] rfPowers)
parameter declaration	rfPowers: antenna output power array;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.15 Obtain the antenna output power (GetOutputPower (bool is16Ants))

function name	Void GetOutputPower (bool is16Ants)
parameter declaration	is16Ants: Using port 16;
returned specification	pass throughThe AntPowerEventArgs parameter in the AntPowerCallback delegate event returns, with the parameter power representing the antenna output power.

2.2.16 Set the echo loss threshold (SetAntConnectionDetector (int detectorSensitivity))

function name	Void SetAntConnectionDetector (int detectorSensitivity)
parameter declaration	detectorSensitivity: the threshold value;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.17 Obtain the echo loss threshold value (GetAntConnectionDetector())

function	Void GetAntConnectionDetector ()
-----------------	----------------------------------

name	
parameter	not have;
declaration	
returned specification	pass throughThe AntConnectionDetectorEventArgs parameter in the AntConnectionDetectorCallback delegate event returns, and the parameter AntConnectionDetector represents the echo loss threshold.

2.2.18 Set the Impinj Monza Lab Quick Read TID Function (SetAndSaveImpinjFastTid (bool useFastTid))

function	Void SetAndSaveImpinjFastTid (bool useFastTid)
name	
parameter	useFastTid: true means use, false means not;
declaration	
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.19 Get the Impinj Monza tag Quick Read TID Function (GetImpinjFastTid())

function	Void GetImpinjFastTid ()
name	
parameter	not have;
declaration	

returned specification	pass throughThe ImpinjFastTidEventArgs parameter in the ImpinjFastTidCallback delegate event returns, with the parameter ImpinjFastTid indicating whether the feature is enabled.
-----------------------------------	---

2.2.20 Set the RF frequency spectrum (SetFrequencyRegion (FrequencyRegion region))

function name	Void SetFrequencyRegion (FrequencyRegion region)
parameter declaration	FrequencyRegion: The RF parameters are as follows; The default system frequency point requires settings: region: Select FCC, ETSI, and CHN in the frequency band; StartFreq: Start frequency point (MHz); EndFreq: End frequency point (MHz); Customization requires the settings: Region: CUSTOMIZE ; StartFreq: Start frequency point (KHz); FreqStep: Channel interval (KHz); FreqQuantity: Number of frequency points; EndFreq: End frequency point (KHz);
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.21 Acquisition of the RF frequency spectrum (GetFrequencyRegion())

function name	Void GetFrequencyRegion ()
parameter declaration	not have;

returned specification	pass throughThe FreqEventArgs parameter in the FrequencyCallback delegate event returns, and the data in the parameter FrequencyRegion: The default system frequency point requires settings: region: Select FCC, ETSI, and CHN in the frequency band; StartFreq: Start frequency point (MHz); EndFreq: End frequency point (MHz); Customization requires the settings: Region: CUSTOMIZE ; StartFreq: Start frequency point (KHz); FreqStep: Channel interval (KHz); FreqQuantity: Number of frequency points;
-------------------------------	---

2.2.22 Set up the RF Link (SetRfLinkProfile (int value))

function name	Void SetRfLinkProfile (int value)
parameter declaration	Value: 0: Tari 25uS; FM0 40KHz 1: Tari 25uS; Miller 4 250KHz 2: Tari 25uS; Miller 4 300KHz 3: Tari 6.25uS; FM0 400KHz
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.23 Get the RF Link (GetRfLinkProfile())

function name	Void GetRfLinkProfile ()
parameter declaration	not have;
returned	pass throughThe RfLinkProfileEventArgs parameter in the RfLinkProfileCallback delegate event returns, and the data in the parameter RfLinkProfile.

specification	0: Tari 25uS; FM0 40KHz 1: Tari 25uS; Miller 4 250KHz 2: Tari 25uS; Miller 4 300KHz 3: Tari 6.25uS; FM0 400KHz
----------------------	---

2.2.24 Set up the E710 RF Link (E710SetRfLinkProfile (E710Profile e710Profile))

function name	Void E710SetRfLinkProfile (E710Profile e710Profile)
parameter declaration	E710Profile : profileId: Link selection is made as follows 0: DSB, Tari 6.25uS, Miller 1 640KHz 1: PR-ASK, Tari 20uS, Miller 4 320KHz 2: PR-ASK, Tari 20uS, Miller 4 250KHz 3: PR-ASK, Tari 20uS, Miller 8 160KHz useDrmMode: Whether to useDrmMode, true is used, and false is not used;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.2.25 Obtain the E710 RF Link (E710GetRfLinkProfile()))

function name	Void GetRfLinkProfile ()
parameter declaration	not have;
returned specification	pass throughThe E710ProfileEventArgs parameter in the E710ProfileCallback delegate event returns, and the data in the parameter E710Profile: E710Profile : profileId: Link selection is made as follows 0: DSB, Tari 6.25uS, Miller 1 640KHz

	1: PR-ASK, Tari 20uS, Miller 4 320KHz 2: PR-ASK, Tari 20uS, Miller 4 250KHz 3: PR-ASK, Tari 20uS, Miller 8 160KHz useDrmMode: Whether to useDrmMode, true is used, and false is not used;
--	--

2.3 Disc storage parameters

Note: Before storage, it is necessary to obtain the RF spectrum of the equipment, otherwise it will cause dll error reporting;

2.3.1 Set disk parameters (setInventoryCfg (InventoryCfg inventoryCfg))

function	Void setInventoryCfg(InventoryCfg inventoryCfg)
name	
parameter declaration	InventoryCfg: The parameters are described as follows: ExecuteTime: Number of runs; CmdInterval: Time interval; Inheritance class: 1. CustomizedSessionTargetInventoryCfg: Custom disk class parameters are described as follows: List <InventoryAntenna>: List of antennas; Session: Enumeration value (S0, S1, S2, S3); Target: enumerated value (A, B); Repeat: Repeats; ReadPhase: Is Phase enabled; PowerSave: whether enabled or not; SelectFlag: Enumeration values (SL0, SL1, SL2, SL3); 2. FastSwitchAntInventoryCfg: Quick inventory class parameters are described as follows: Interval: Interantenna delay (ms); SelectFlag: Enumeration values (SL0, SL1, SL2, SL3); Repeat: Repeats; Session: Enumeration value (S0, S1, S2, S3); Target: enumerated value (A, B); Optimize class: onlybuffer: false; times: times; Ongoing: Continuous times; TargetQuantity : ReadPhase: Is Phase enabled; List <int> TempPowers: Temporary power list;
returned specification	not have;

2.3.2 Custom drive mode

(CustomizedSessionTargetInventory

(CustomizedSessionTargetInventoryCfg cfg))

function name	Void CustomizedSessionTargetInventory(CustomizedSessionTargetInventoryCfg cfg)
parameter declaration	CustomizedSessionTargetInventoryCfg: Custom disk class parameters are described as follows: List <InventoryAntenna>: List of antennas; Session: Enumeration value (S0, S1, S2, S3); Target: enumerated value (A, B); Repeat: Repeats; ReadPhase: Is Phase enabled; PowerSave: whether enabled or not; SelectFlag: Enumeration values (SL0, SL1, SL2, SL3);
returned specification	pass throughTagRead, TagReadDataEventArgs parameter in delegate event, data in parameter TagData: bArrPC: PC data; bArrEPC: EPC data; Antenna: antenna; The bRssi: the Rssi value; FrequencyRegion: spectrum data; iFrequency: frequency point index; iReadCount: Total number of reads;

2.3.3 Quick Drive-Mode (FastSwitchAntInventory

(FastSwitchAntInventoryCfg cfg))

function name	Void FastSwitchAntInventory(FastSwitchAntInventoryCfg cfg)
parameter declaration	FastSwitchAntInventoryCfg: Quick inventory class parameters are described as follows: Interval: Interantenna delay (ms); SelectFlag: Enumeration values (SL0, SL1, SL2, SL3); Repeat: Repeats;

	Session: Enumeration value (S0, S1, S2, S3); Target: enumerated value (A, B); Optimize class: onlybuffer: false; times: times; Ongoing: Continuous times; TargetQuantity : ReadPhase: Is Phase enabled; List <int> TempPowers: Temporary power list;
returned specification	pass throughTagRead, TagReadDataEventArgs parameter in delegate event, data in parameter TagData: bArrPC: PC data; bArrEPC: EPC data; Antenna: antenna; The bRssi: the Rssi value; FrequencyRegion: spectrum data; iFrequency: frequency point index; iReadCount: Total number of reads;

2.4 Label operation

2.4.1 Set the selected label (SetAccessEpcMatch (EpcMatch epcMatch))

function name	Void SetAccessEpcMatch (EpcMatch epcMatch)
parameter declaration	EpcMatch class: incoming requires selected EPC data; Byte [] epc: epc array; Int epcLen: epc length; Bool match: true is a match and false is a unmatched;
returned specification	pass throughThe SettingEventArgs parameter in the SettingStatusCallback delegate event returns, with the ErrorCode in the parameter status represented.

2.4.2 Get the selected label (GetAccessEpcMatch())

function name	Void GetAccessEpcMatch ()
parameter declaration	not have;
returned specification	pass throughThe EpcMatchEventArgs parameter in the EpcMatchCallback delegate event returns, with the EPC in the parameter EpcMatch represented.

2.4.3 Read label (Read (MemBank memBank, int wordAddr, int wordCnt, byte [] accessPwd))

function name	Void Read (MemBank memBank, int wordAddr, int wordCnt, byte[] accessPwd)
parameter declaration	The MemBank class, enumerated: 0x00 RESERVED ; 0x01 EPC ; 0x02 TID ; 0x03 USER ; wordAddr: Read the data first address; wordCnt: Read the data length; accessPwd: Lab access password;
returned specification	pass throughThe TagOpResultEventArgs parameter in the TagOpResultCallback delegate event returns, and the parameter tagOps class indicates: TagData: label data; ErrorCode: The error was returned;

2.4.4 Read the label for multiple regions (Read(int reservedAddr,int reservedLen,int tidAddr,int tidLen,int userAddr,int userLen,byte [] accessPwd,Session session,Target target,ReadMode readMode,int timeout))

function name	Void Read (int reservedAddr, int reservedLen, int tidAddr, int tidLen, int userAddr, int userLen, byte[] accessPwd, Session session, Target target, ReadMode readMode, int timeout)
parameter declaration	reservedAddr: Reservation first address; reservedLen: Reserve data length; tidAddr: First address of Tid area; tidLen: Data length of the Tid area; userAddr: First address of the user area; userLen: Data length of the user area; accessPwd: Lab access password; The Session enumeration class:

	S0、S1、S2、S3 The Target enumeration class: A、B The readMode enumeration class: 0x00: Single-label mode, no Session control; 0x01: Single-label mode, with Session control; 0x02: Multi-label mode, with Session control; Timeout: Label response timeout control
returned specification	pass throughThe TagOpResultEventArgs parameter in the TagOpResultCallback delegate event returns, and the parameter tagOps class indicates: TagData: label data; ErrorCode: The error was returned;

2.4.5 Write a label (Write (byte [] accessPwd, MemBank membank, int wordAddr, int wordCnt, byte [] writeData))

function name	Void Write (byte[] accessPwd, MemBank memBank, int wordAddr, int wordCnt, byte[] writeData)
parameter declaration	accessPwd: Lab access password; The MemBank class, enumerated: 0x00 RESERVED ; 0x01 EPC ; 0x02 TID ; 0x03 USER ; wordAddr: Write to the data first address; wordCnt: Write to the data length; writeData: The written data;
returned specification	pass throughThe TagOpResultEventArgs parameter in the TagOpResultCallback delegate event returns, and the parameter tagOps class indicates: TagData: label data; ErrorCode: The error was returned;

2.4.6 Block (BlockWrite (byte [] accessPwd, MemBank membank, int wordAddr, int wordCnt, byte [] writeData))

function	Void Block Write (byte[] accessPwd , MemBank memBank, int wordAddr, int wordCnt, byte[] writeData)
-----------------	---

name	
parameter declaration	accessPwd: Lab access password; The MemBank class, enumerated: 0x00 RESERVED ; 0x01 EPC ; 0x02 TID ; 0x03 USER ; wordAddr: Write to the data first address; wordCnt: Write to the data length; writeData: The written data;
returned specification	pass throughThe TagOpResultEventArgs parameter in the TagOpResultCallback delegate event returns, and the parameter tagOps class indicates: TagData: label data; ErrorCode: The error was returned;

2.4.7 Lock label (Lock (byte [] accessPwd, LockMemBank membank, LockType lockType))

function name	Void Lcok (byte[] accessPwd, LockMemBank memBank, LockType lockType)
parameter declaration	accessPwd: Lab access password; The LockMemBank class, enumerated: 0x01: USER; 0x02: TID; 0x03: EPC; 0x04: ACCESS_PASSWORD ; 0x05: KILL_PASSWORD ; The LockType class, enumerated: 0x00: UNLOCK ; 0x01: LOCK ; 0x02: PERMANENT_UNLOCK ; 0x03: PERMANENT_LOCK ;
returned specification	pass throughThe TagOpResultEventArgs parameter in the TagOpResultCallback delegate event returns, and the parameter tagOps class indicates: TagData: label data; ErrorCode: The error was returned;

2.4.8 Destroy tags (Kill (byte [] killPwd))

function name	Void Kill(byte[] killPwd))
parameter declaration	killPwd: destroy the password;
returned specification	pass throughThe TagOpResultEventArgs parameter in the TagOpResultCallback delegate event returns, and the parameter tagOps class indicates: TagData: label data; ErrorCode: The error was returned;

2.5 Error code

price	name	Descriptions
0x10	command_success	The command was completed successfully
0x11	command_fail	Command execution failed
0x20	mcu_reset_error	CPU reset error
0x21	cw_on_error	Opens CW error
0x22	antenna_missing_error	The antenna is not connected
0x23	write_flash_error	Write a Flash Error
0x24	read_flash_error	Read the Flash Error
0x25	set_output_power_error	Set the transmit power in error
0x31	tag_inventory_error	Discap label error
0x32	tag_read_error	Read label error
0x33	tag_write_error	Write label error
0x34	tag_lock_error	Lock tag error
0x35	tag_kill_error	Inactivation label error
0x36	no_tag_error	No operation label error

0x37	inventory_ok_but_access_fail	Disached successfully but access failed
0x38	buffer_is_empty_error	The cache is empty
0x3C	nxp_custom_command_fail	The NXP chip custom instruction failed
0x40	access_or_password_error	Access tag error or access password error
0x41	parameter_invalid	Invalid parameters
0x42	parameter_invalid_wordCnt_too_long	The wordCnt parameter exceeds the specified length
0x43	parameter_invalid_membank_out_of_range	The MemBank parameter is out of range
0x44	parameter_invalid_lock_region_out_of_range	The Lock data zone parameter is out of range
0x45	parameter_invalid_lock_action_out_of_range	The LockType parameter is out of range
0x46	parameter_reader_address_invalid	The reader address is invalid
0x47	parameter_invalid_antenna_id_out_of_range	The Antenna_id is out of range
0x48	parameter_invalid_output_power_out_of_range	Output power parameter is out of range
0x49	parameter_invalid_frequency_region_out_of_range	The RF specification area parameters are out of range
0x4A	parameter_invalid_baudrate_out_of_range	The Baud rate parameter is out of range
0x4B	parameter_beeper_mode_out_of_range	The buzzer setting parameter is out of range
0x4C	parameter_epc_match_len_too_long	EPC matching length crossed
0x4D	parameter_epc_match_len_error	EPC match length error
0x4E	parameter_invalid_epc_match_mode	The EPC matching parameter is

		out of range
0x4F	parameter_invalid_frequency_range	Wrong frequency range setting parameter
0x50	fail_to_get_RN16_from_tag	Unable not receive RN16 for label
0x51	parameter_invalid_drm_mode	DRM setting parameter error
0x52	pll_lock_fail	The PLL cannot be locked
0x53	rf_chip_fail_to_response	The RF chip is not responsive
0x54	fail_to_achieve_desired_output_power	The output does not reach the specified output power
0x55	copyright_authentication_fail	Copyright certification has failed
0x56	spectrum_regulation_error	Spectrum specification is set in error
0x57	output_power_too_low	The output power is too low
0xEE	fail_to_get_rf_port_return_loss	Measurement of the echo loss has failed